

**FILIÈRE : CONCEPTION ET
DÉVELOPPEMENT DE LOGICIELLE**

MINI-PROJET

“TASKFLOW”

PRÉPARER PAR :

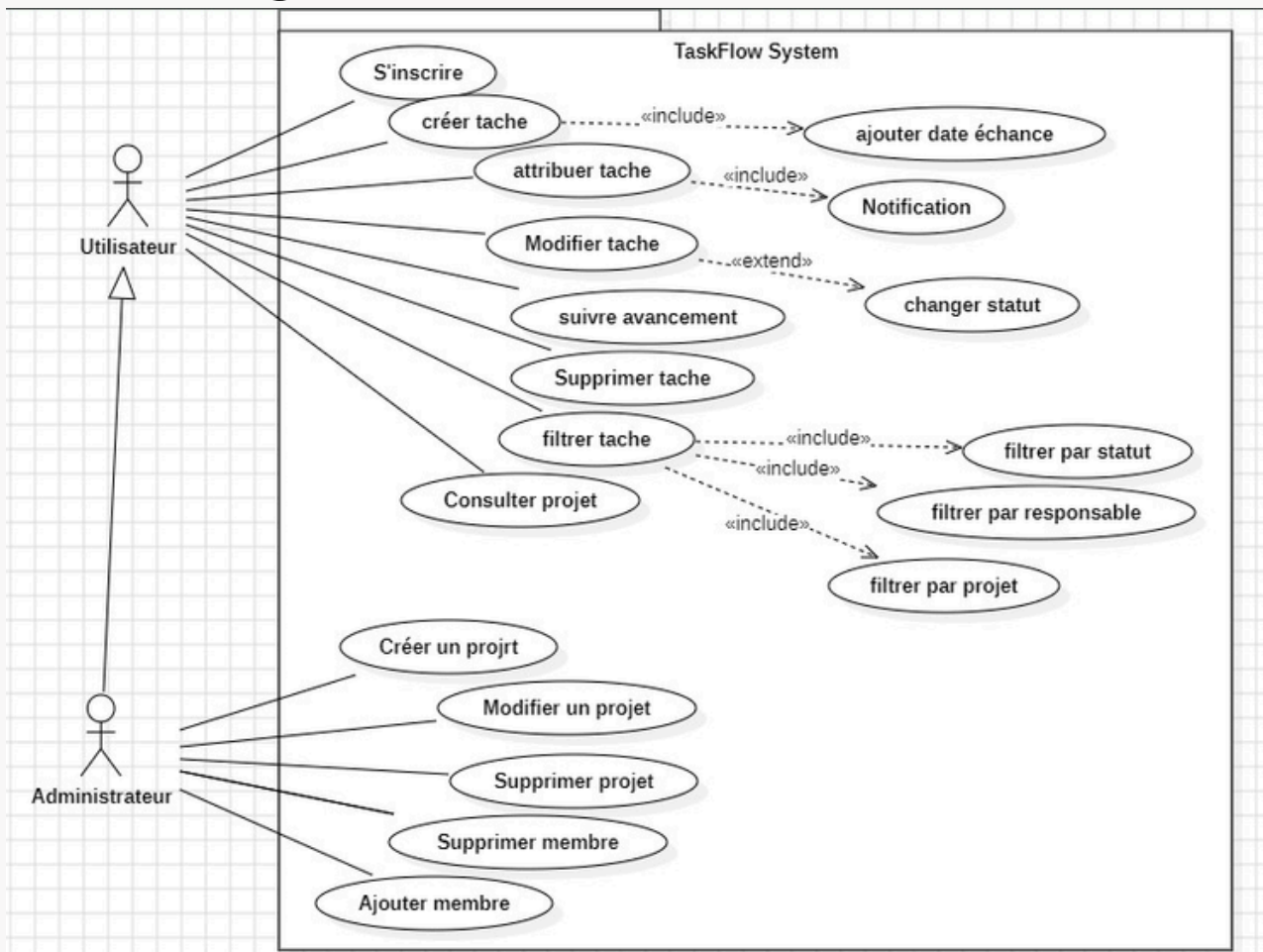
HAJAR OUAAZZAN

IKHLAS TEYEB

ILYAS IBRAHIMI

MAROUA LAMGHARI

diagramme de cas d'utilisation



Ce diagramme décrit les fonctionnalités du système

TaskFlow à travers deux acteurs :

L'Utilisateur : Il peut s'inscrire, créer et attribuer des tâches. Il a aussi la possibilité de modifier, supprimer et filtrer ses tâches (par statut, responsable ou projet) pour suivre l'avancement.

L'Administrateur : En plus des droits de l'utilisateur, il gère les projets (création, modification, suppression) et s'occupe de la gestion des membres (ajouter ou supprimer).

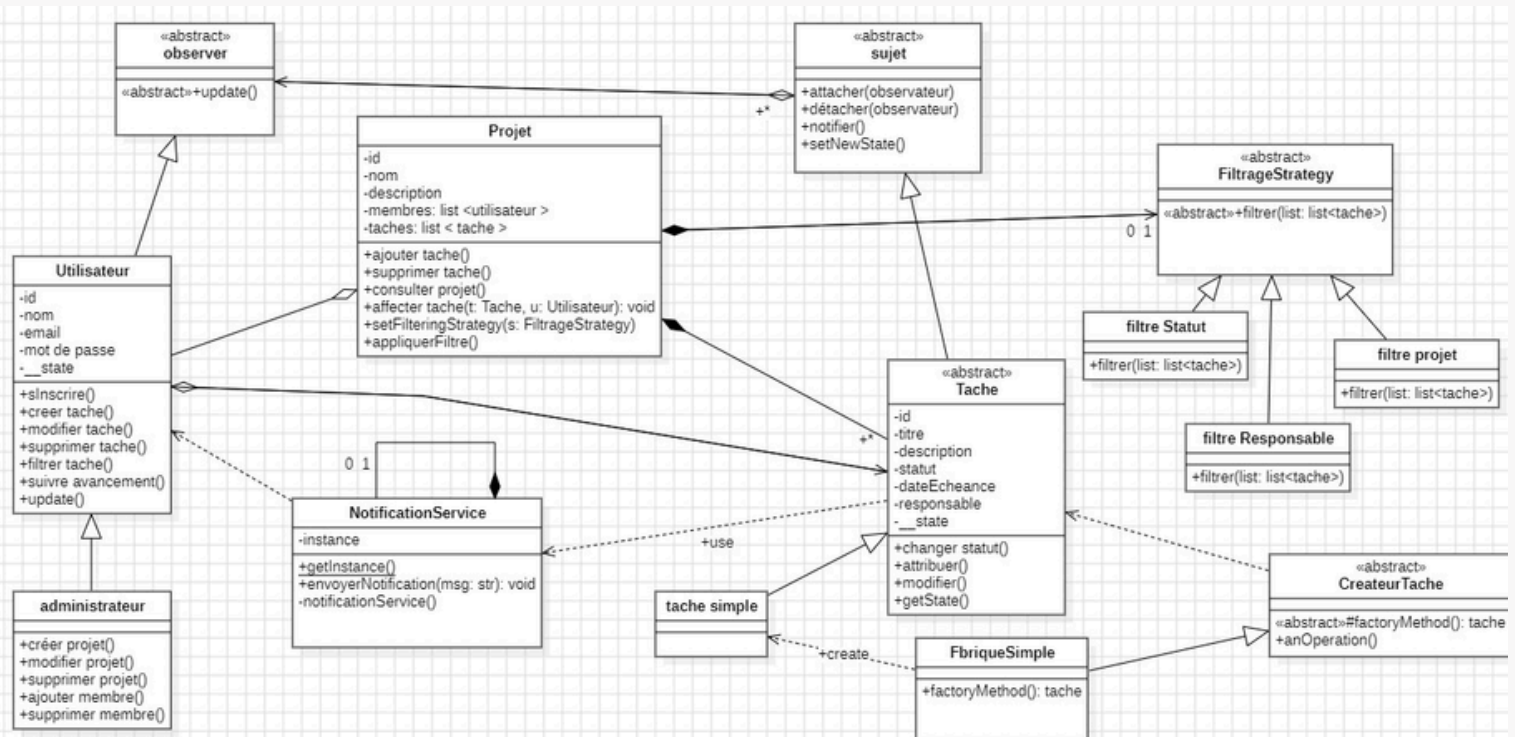
Relations clés :

Inclusion («include») : Par exemple, la création d'une tâche nécessite obligatoirement une date d'échéance.

Extension («extend») : La modification d'une tâche permet, si besoin, de changer son statut.

Héritage : L'administrateur hérite de toutes les fonctionnalités de l'utilisateur.

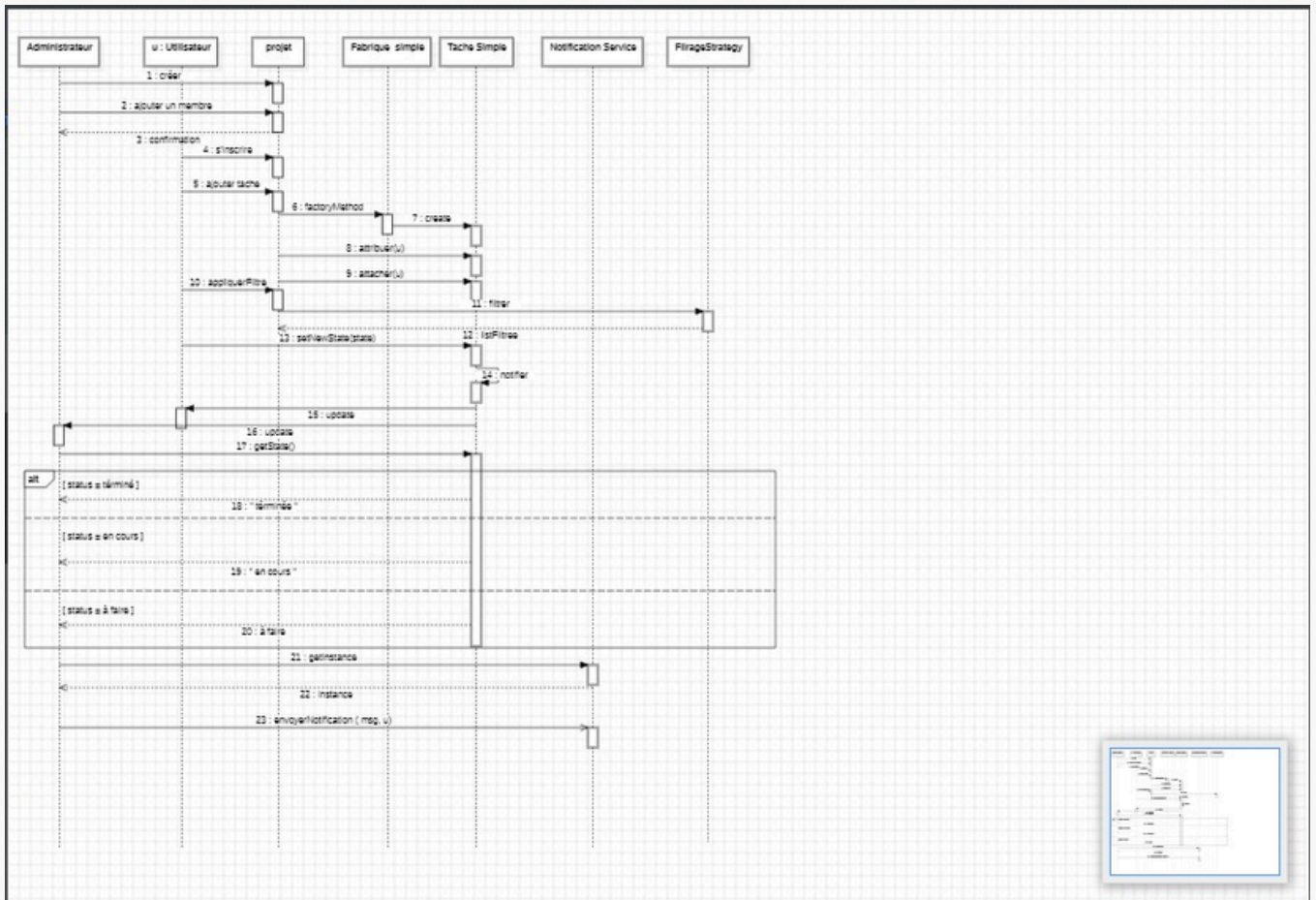
diagramme de Classe



- Ce diagramme montre la structure technique du système et comment les objets communiquent entre eux. Voici les points principaux :

- 1. Les classes de base :
- Utilisateur & Administrateur : L'administrateur hérite de l'utilisateur. Ils gèrent les informations personnelles (nom, email, mot de passe).
- Projet : Contient une liste de tâches et de membres. C'est l'élément central qui regroupe tout.
- Tache : Représente une action à faire avec un titre, une date d'échéance et un statut.
- 2. Les Design Patterns (Modèles de conception) utilisés :
- Le diagramme utilise des solutions intelligentes pour organiser le code :
- Pattern Observer (Observer/Sujet) : Utilisé pour le système de notifications. Quand une tâche change, les utilisateurs concernés sont informés automatiquement.
- Pattern Strategy (FiltrageStrategy) : Permet de changer la façon de filtrer les tâches (par statut, par responsable ou par projet) de manière flexible.
- Pattern Factory (CreateurTache) : Sert à créer des objets "Tache" de façon organisée.
- Pattern Singleton (NotificationService) : Garantit qu'il n'y a qu'une seule instance du service de notification dans toute l'application.
- 3. Les relations :
- Héritage : L'administrateur est un utilisateur spécialisé.
- Composition/Agrégation : Un projet est composé de plusieurs tâches et membres

diagramme de Séquence



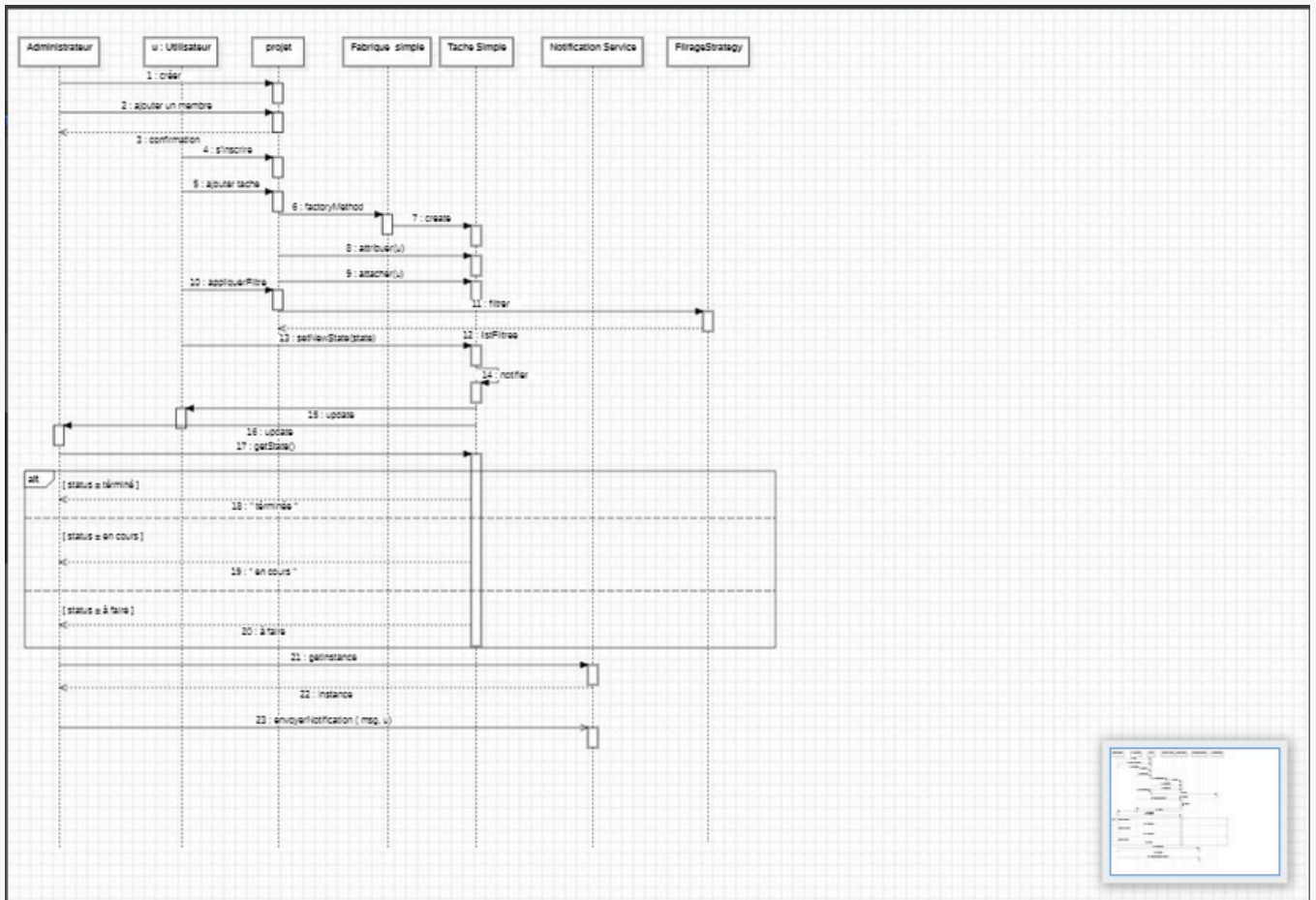
Ce diagramme détaille le fonctionnement dynamique du système à travers trois étapes clés :

Gestion du Projet : L'administrateur crée le projet et ajoute les membres, tandis que l'utilisateur s'y inscrit pour participer.

Cycle des Tâches : Le système utilise le pattern Factory pour créer les tâches, les attribuer, et le pattern Strategy pour permettre à l'utilisateur de les filtrer.

Notifications & Patterns : En cas de changement de statut (vérifié par un bloc alt), le pattern Observer déclenche une mise à jour. Le NotificationService (via un Singleton) envoie alors une alerte aux personnes concernées.

diagramme de Séquence



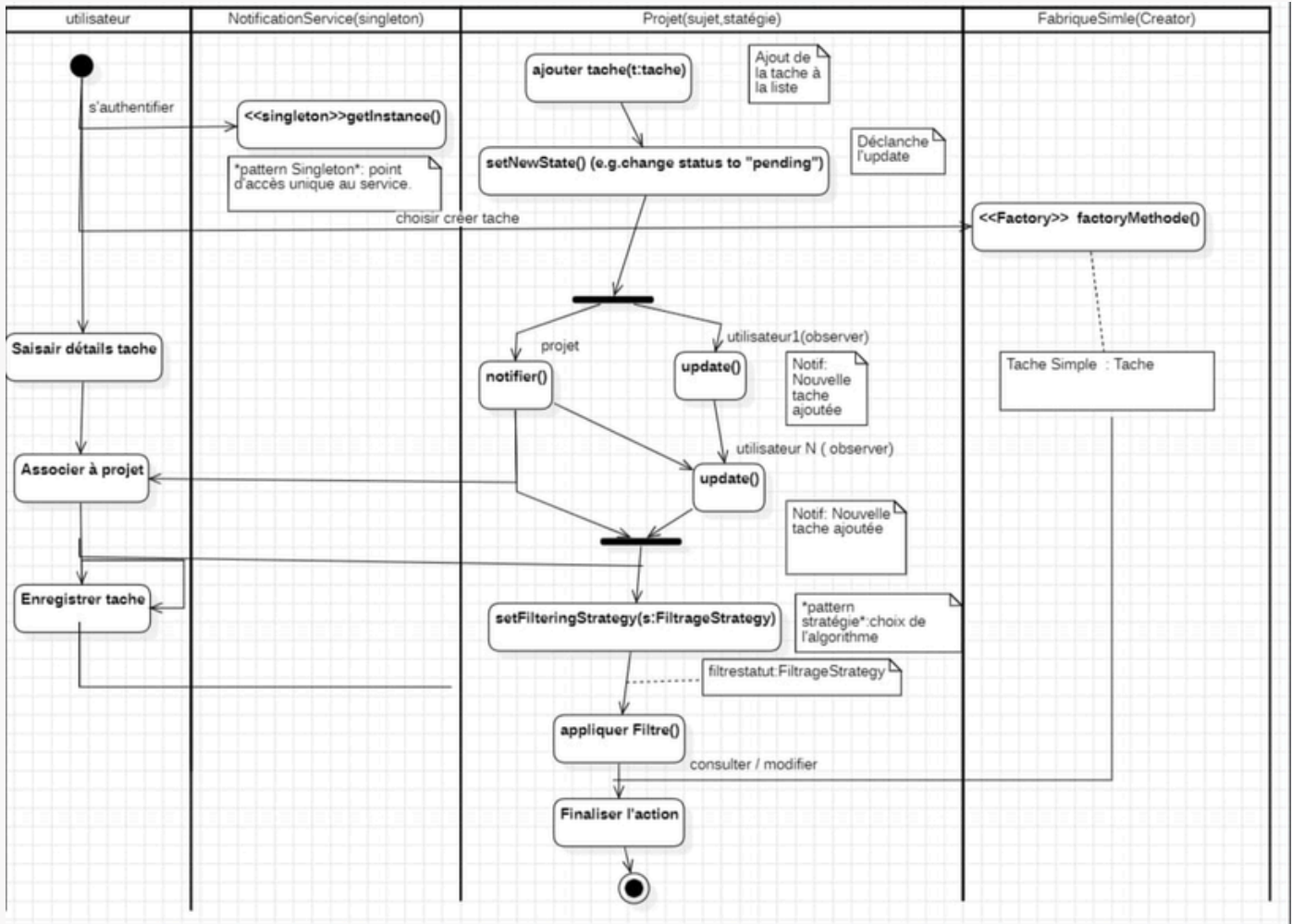
Ce diagramme détaille le fonctionnement dynamique du système à travers trois étapes clés :

Gestion du Projet : L'administrateur crée le projet et ajoute les membres, tandis que l'utilisateur s'y inscrit pour participer.

Cycle des Tâches : Le système utilise le pattern Factory pour créer les tâches, les attribuer, et le pattern Strategy pour permettre à l'utilisateur de les filtrer.

Notifications & Patterns : En cas de changement de statut (vérifié par un bloc alt), le pattern Observer déclenche une mise à jour. Le NotificationService (via un Singleton) envoie alors une alerte aux personnes concernées.

diagramme d'Activité



- Ce diagramme représente le flux de travail (workflow) lors de la création et de la gestion d'une tâche. Il montre comment les actions s'enchaînent entre l'utilisateur et les différents services :
- Authentification et Initialisation : L'utilisateur commence par s'authentifier. Le système utilise le Singleton (getInstance) pour préparer le service de notification.
- Création de la Tâche : L'utilisateur saisit les détails. Le système fait appel à la Fabrique (Factory) pour créer l'objet tâche.
- Notification (Observer) : Une fois la tâche ajoutée au projet, l'état change (setNewState). Cela déclenche automatiquement une mise à jour (update) pour informer tous les observateurs (utilisateurs concernés).
- Filtrage et Finalisation : L'utilisateur peut choisir une stratégie de filtrage (Strategy Pattern) pour organiser l'affichage avant de finaliser l'action.

IMPLEMENTATION

```
1 package models;
2 import java.util.Date;
3
4 /**
5  * Classe représentant une tâche dans le projet.
6  */
7 public class Tache {
8     private int id;
9     private String titre;
10    private String description;
11    private String statut;
12    private Date dateEcheance;
13    private Utilisateur responsable;
14
15    public void changerStatut(String nouveauStatut) {
16        this.statut = nouveauStatut;
17    }
18
19    public String getTitre() { return titre; }
20 }
```

```
1 package models;
2
3 public class TacheSimple extends Tache {
4
5     public TacheSimple() {
6         this.statut = "Nouvelle";
7     }
8 }
```

```
1 package models;
2 import patterns.Observer;
3
4 /**
5  * Représente un utilisateur du système
6  */
7 public class Utilisateur implements Observer {
8     protected int id;
9     protected String nom;
10    protected String email;
11    protected String motDePasse;
12
13    public void sInscrire() { /* Code */ }
14
15    @Override
16    public void update() {
17        System.out.println(nom + " a reçu une mise à jour !");
18    }
19
20    public String getNom() { return nom; }
21 }
```

```
1 public class Mainfest {
2     public static void main(String[] args) {
3
4         // 1. Tester le Singleton (Notification)
5         NotificationService logger = NotificationService.getInstance();
6         logger.envoyerNotification("msg", "Application démarrée avec succès.");
7
8         // 2. Tester la Factory (Création de tâche)
9         FabriqueSimple fabrique = new FabriqueSimple();
10        models.Tache maTache = fabrique.factoryMethod();
11        System.out.println("Tache créée via Factory.");
12
13        // 3. Tester l'Observer
14        Utilisateur user1 = new Utilisateur();
15        user1.setNom("Ahmed");
16
17        // Simulation d'un projet
18        Projet p = new Projet();
19        // ... Ajouter les tâches de test
20
21        System.out.println("---- Test terminé ----");
22    }
23 }
```

```
1 package patterns;
2
3 /**
4  * Interface pour l'observateur (Pattern Observer)
5  */
6 public interface Observer {
7     void update();
8 }
9
10
11
```

```
1 package patterns;
2
3 /**
4  * Service de notification utilisant le Pattern Singleton
5  */
6 public class NotificationService {
7     private static NotificationService instance;
8
9     private NotificationService() { /* Constructeur privé */ }
10
11     public static NotificationService getInstance() {
12         if (instance == null) {
13             instance = new NotificationService();
14         }
15         return instance;
16     }
17
18     public void envoyerNotification(String msg) {
19         System.out.println("Notification: " + msg);
20     }
21 }
```

```
1 package patterns;
2
3 import java.util.List;
4 import models.Tache;
5
6 /**
7  * Interface pour la stratégie de filtrage (Pattern Strategy)
8  */
9 public interface FiltrageStrategy {
10     List<Tache> filtrer(List<Tache> taches);
11 }
```

IMPLEMENTATION

Le code représente une application de gestion de tâches appelée TaskFlow. Elle permet de gérer des projets, des tâches et des utilisateurs dans un environnement collaboratif.

Modèle (models)

Utilisateur : représente un utilisateur du système et peut recevoir des notifications grâce au pattern Observer.

Projet : contient une liste de tâches et de membres, et permet de filtrer les tâches avec une stratégie (Strategy Pattern).

Tache : représente une tâche avec un titre, une description, un statut, une date d'échéance et un responsable.

Patterns utilisés

Observer : permet de notifier automatiquement les utilisateurs lors des changements.

Strategy : permet de changer dynamiquement la méthode de filtrage des tâches.

Factory Method : utilisé pour créer des objets Tache de manière flexible.

Singleton : utilisé pour le service de notification afin d'avoir une seule instance.

Services

ProjetService : gère les opérations principales comme l'affectation des tâches aux utilisateurs et l'envoi des notifications.

Utilitaires

DateUtils : permet de formater les dates dans un format lisible.

Test

MainTest : classe principale qui teste les différentes fonctionnalités du système (Singleton, Factory, Observer).

tayebikhlass-dev/**Task-Flow**



1

Contributor



0

Issues



0

Stars



0

Forks



tayebikhlass-dev/**Task-Flow**

Contribute to tayebikhlass-dev/Task-Flow development by creating an account on GitHub.



GitHub

<https://github.com/tayebikhlass-dev/Task-Flow>

Le projet contient plusieurs dossiers organisés pour faciliter la compréhension et le développement :

Dossier Conception UML

Ce dossier contient les différents diagrammes UML du projet :

Diagramme de cas d'utilisation (Use Case)

Diagramme de classes

Diagramme de séquence

Diagramme d'activité

Dossier Code Source

Ce dossier contient l'implémentation du projet en Java avec une architecture organisée en plusieurs packages :

models : contient les classes principales du système

pattern : implémentation des Design Patterns utilisés

services : gestion de la logique métier

utils : classes utilitaires

test : classes de test et validation

Rapport

Le projet contient également un rapport détaillé expliquant :

l'analyse des besoins

la conception UML

les choix techniques

l'implémentation

les tests réalisés

technologies utilisées

Java

IntelliJ IDEA

Git & GitHub

UML